Samsung Innovation Campus

Batch 8

Big Data Track

Home Credit Default Risk

End-to-End Big Data Engineering Pipeline

Facilitator: Bassmala Hossam

Submitted by:

Student Name	Group
Mohamed Anas Wahban	SIC_BD 801
Mohamed Samir Abdallah	SIC_BD 801
Mohamed Fouad Elgohary	SIC_BD 801
Shenouda Aboelrose	SIC_BD 801

September 2026

Contents

1. Project Overview & Objectives

1.1. Why This Project?

The Home Credit dataset presents significant Data Engineering challenges that mirror real-world industry scenarios. The project involves processing multiple large relational tables containing millions of records. Our primary objective is to build a scalable and automated Big Data ETL pipeline that seamlessly transitions from raw operational databases (MySQL) to a highly optimized, distributed Data Warehouse (Hive/HDFS). Ultimately, the core purpose of this engineered dataset is to serve as a clean, reliable foundation for an AI model to predict default probabilities, alongside supporting Business Intelligence (BI) reporting.

1.2. Pipeline Architecture

The data engineering workflow is structured into the following sequential phases, ensuring strict multi-user permission separation and data integrity across the cluster:

- **Data Modeling & Star Schema Design:** Designing a robust Data Model to prevent Cartesian joins and cluster memory overloads, structuring the dataset into a Dimension table (`Dim_Customer`) for BI slicing and a Fact table (`Fact_Loan`) as a feature matrix for Machine Learning.
- **Database Setup & Bulk Ingestion:** Loading the raw CSV datasets (`application_train`, `bureau`, `previous_application`, and `installments_payments`) into a relational MySQL database hosted inside a Docker container.
- **Big Data Ingestion (HDFS & Apache Sqoop):** Extracting the raw relational tables from MySQL directly into the Hadoop Distributed File System (HDFS) landing zones using Apache Sqoop MapReduce tasks.
- **Distributed Processing & Data Selection (PySpark):** Leveraging PySpark on the Hadoop/YARN cluster to initialize schemas, read raw records, and explicitly select the required analytical columns.
- **Data Cleaning & Staging Layer:** Applying rigorous data quality checks including dynamic empty string conversion, deduplication, null filtering, and business anomaly correction, followed by exporting the results to a high-performance Staging Layer in Parquet format.
- **Feature Engineering:** Performing distributed grouped aggregations at the customer grain (`SK_ID_CURR`) to maintain a strict one-to-one relationship, alongside cross-table feature derivation to compute complex financial metrics.

- **Data Warehouse Serving Layer (Star Schema):** Transitioning the unified engineered dataset into the finalized Star Schema architecture, exporting the data to HDFS in Parquet format with Snappy compression, and registering the partitioned tables within the Hive Metastore for immediate ML and BI consumption.

2. Phase 1: Data Modeling & Star Schema Design

2.1. Original Data Architecture & Challenges

The original dataset was provided in a highly normalized, snowflake-like relational structure. As illustrated in Figure 1, the core application data (`application_train.csv`) is linked to multiple secondary tables (e.g., `bureau.csv`, `previous_application.csv`, `installments_payments.csv`) via primary and foreign keys such as `SK_ID_CURR` and `SK_ID_PREV`.

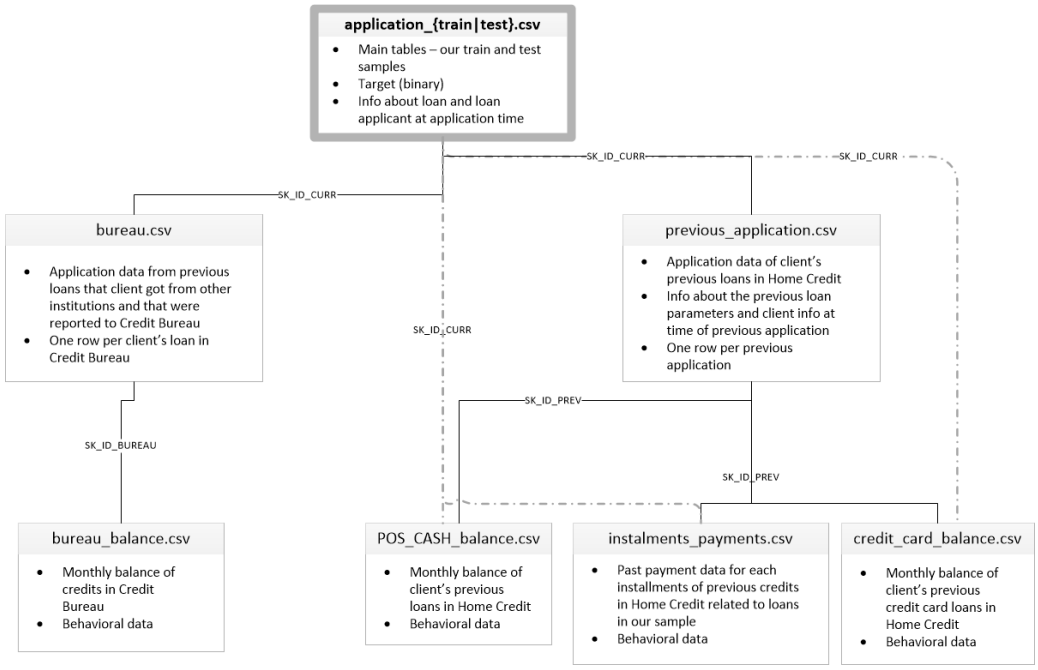

Figure 1: Original Home Credit Relational Schema representing transactional loan data.

The primary challenge with this architecture in a Big Data analytics context is the **Many-to-Many** relationship inherent in the historical tables. A single customer (`SK_ID_CURR`) might have dozens of previous applications and hundreds of installment payments.

2.2. Aggregation Strategy: Preventing Data Explosion

To transition this transactional structure into an analytical Star Schema, a strict data modeling rule was established: **Direct joins between the main application table and the secondary historical tables are strictly prohibited**.

Attempting a direct join would result in a massive Cartesian product (Data Explosion), creating duplicate records for the primary application and immediately crashing the PySpark pipeline due to cluster memory overload. To solve this, an aggregation strategy was implemented:

- **Pre-Join Grouping:** Every secondary table (`previous_application`, `installments_payments`, `bureau`) must undergo a `GroupBy("SK_ID_CURR")` operation first.
- **Grain Enforcement:** This guarantees that the output DataFrame yields exactly **ONE row per customer** before executing the final Left Join onto the main fact table.

2.3. Final Star Schema Design

The transformed architecture is divided into two highly optimized tables: a Dimension table for descriptive Business Intelligence (BI) slicing, and a central Fact table acting as the primary feature matrix.

1. Dimension Table: `Dim_Customer`

This table isolates the descriptive, categorical attributes of the customer. It is primarily utilized for interactive dashboard filters (Slicers) and demographic segmentation without cluttering the mathematical model.

Column Name	Status	Source Table
SK_ID_CURR	PK (1 Row/Cust)	application_train
CODE_GENDER	Existing	application_train
FLAG_OWN_CAR	Existing	application_train
NAME_CONTRACT_TYPE	Existing	application_train
OCCUPATION_TYPE	Existing	application_train
NAME_EDUCATION_TYPE	Existing	application_train
NAME_FAMILY_STATUS	Existing	application_train
NAME_HOUSING_TYPE	Existing	application_train
NAME_INCOME_TYPE	Existing	application_train
FLAG_OWN_REALTY	Existing	application_train

2. Fact Table: `Fact_Loan`

This is the central Feature Matrix. It contains raw financial numbers, advanced engineered features, and a dynamically generated surrogate key.

A. Raw Core Columns:

Column Name	Status	Source Table
SK_ID_CURR	Existing (Join Key)	application_train
loan_key	Auto-Increment ID	monoton_increas_id
TARGET	Existing (Label)	application_train
NAME_CONTRACT_TYPE	Existing (Partition)	application_train
AMT_INCOME_TOTAL	Existing	application_train
AMT_CREDIT	Existing	application_train
AMT_ANNUITY	Existing	application_train
AMT_GOODS_PRICE	Existing	application_train
DAYS_EMPLOYED	Existing	application_train
DAYS_BIRTH	Existing (Numeric)	application_train

B. Engineered Features (Derived via PySpark):

These features were synthetically generated to capture customer financial behavior, repayment discipline, and external credit exposure.

Column Name	Source	Business Rationale
DTI	app_train	Measures financial burden; high ratio indicates struggle to repay.
Annuity_to_Income	app_train	Assesses monthly installment affordability.
LTV	app_train	Evaluates collateral risk; high LTV means loan exceeds asset value.
Employed_to_Age	app_train	Indicates employment stability relative to age.
Prev_App_Count	prev_app	Customer's historical reliance on Home Credit.
Approved_App_Ratio	prev_app	Reflects historical creditworthiness and success rate.
Refused_App_Ratio	prev_app	Highlights past rejections (strong risk indicator).
Avg_Prev_Credit	prev_app	Baseline for the customer's typical borrowing size.
Total_Days_Past_Due	installs	Quantifies severity of historical payment delays.

Continued on next page

Column Name	Source	Business Rationale
Num_Late_Payments	installs	Indicates frequency of poor repayment behavior.
Avg_Days_Past_Due	installs	Typical delay length, differentiating chronic lateness.
Max_Days_Past_Due	installs	Identifies the worst-case historical default behavior.
Total_Underpaid	installs	Highlights situations of consistent underpayment.
Payment_Ratio	installs	Measures overall repayment discipline (¡ 1.0 = underpaying).
Total_External_Debt	bureau	Total financial obligation outside of Home Credit.
Total_External_Overdue	bureau	Exact amount currently defaulted with other lenders.
Max_External_Overdue	bureau	Highlights the worst external default.
Bureau_Active_Loans	bureau	Customer's active exposure to external lenders.
Bureau_Credit_Count	bureau	Total external credit accounts on record.
Active_Credit_Ratio	bureau	Ratio of active obligations to total historical credit lines.
Debt_to_Credit_Bureau	bureau	Indicates credit utilization across all external accounts.
External_DTI	bur+app	Total external debt burden against actual income.
Total_Overall_Debt	bur+app	Complete combined debt picture (Internal + External).

3. Phase 2: Database Setup & Bulk Ingestion

3.1. Environment Strategy & Prerequisites

To simulate a real-world production environment where data engineers extract data from operational transactional databases, we hosted a MySQL instance inside a Docker container named `my_local_mysql`. This containerized approach ensures environmental consistency, isolates dependencies, and provides a realistic source system for our subsequent Hadoop ingestion pipeline.

Before initializing the database, the raw CSV files required for the project (`application_train.csv`, `bureau.csv`, `previous_application.csv`, and `installments_payments.csv`) were securely transferred from the host machine into the container's temporary directory (`/tmp/raw/`).

Listing 1: Transferring raw data to the MySQL Docker container

```
# Copy the raw dataset directory into the container's /tmp path
docker cp raw my_local_mysql:/tmp/raw
```

Figure 2: Terminal output demonstrating the successful transfer of raw CSV files to the Docker container.

3.2. Strict Schema Definition (DDL)

Data integrity is paramount in data engineering. Instead of relying on automated schema inference which often results in incorrect data types (e.g., assigning string types to numeric IDs), a strict Data Definition Language (DDL) script (`init_db.sql`) was executed.

The schema enforced precise data types (e.g., `DECIMAL(15, 2)` for financial amounts, `INT` for identifiers) and established Primary Keys to guarantee uniqueness at the database level:

- `application_train`: 122 Columns, PK: `SK_ID_CURR`.
- `bureau`: 17 Columns, PK: `SK_ID_BUREAU`.
- `previous_application`: 37 Columns, PK: `SK_ID_PREV`.
- `installments_payments`: 8 Columns, transactional records tied to `SK_ID_CURR` and `SK_ID_PREV`.

3.3. High-Performance Bulk Ingestion

Standard `INSERT` statements are highly inefficient for millions of rows. To achieve maximum throughput, the ingestion process utilized MySQL's `LOAD DATA LOCAL INFILE` command. This bulk-loading mechanism bypasses typical SQL parsing overhead, allowing gigabytes of raw CSV data to be mapped directly into the defined tables in minutes. The script also automatically converts empty string values to structural `NULL`s to maintain analytical accuracy.

Listing 2: Bulk Ingestion using LOAD DATA INFILE

```
LOAD DATA LOCAL INFILE '/tmp/raw/application_train.csv'
INTO TABLE application_train
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
```

```
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

3.4. Execution & Resource Cleanup

The entire schema creation and bulk ingestion process was triggered externally via a single `docker exec` command, utilizing the `--local-infile=1` flag to permit local file reading.

Listing 3: Executing the SQL script and cleaning up resources

```
# Execute the bulk ingestion script
docker exec -i my_local_mysql mysql -u root -prootpassword --local-
    infile=1 < init_db.sql

# Clean up temporary raw data to release container storage
docker exec my_local_mysql rm -rf /tmp/raw
```

Once the data was successfully verified inside the MySQL tables, the temporary CSV files located in `/tmp/raw` were deleted. This cleanup prevents Docker container bloat and adheres to best practices for container storage management.

Figure 3: MySQL execution logs and table row counts verifying successful bulk ingestion.

4. Phase 3: Big Data Ingestion (HDFS & Apache Sqoop)

4.1. Architectural Context & Network Bridging

In enterprise environments, operational data rarely resides on the same infrastructure as the Big Data analytics cluster. To simulate this real-world topology, our operational MySQL database is hosted on the Docker host machine, while the Hadoop ecosystem operates inside a separate Virtual Machine (VM).

Before initiating the extraction process, the host IP address (e.g., `10.29.51.13`) was identified to establish a network bridge. This IP was used to construct a valid JDBC connection string (`jdbc:mysql://<HOST_IP>:3306/home_credit`) so that Apache Sqoop could communicate seamlessly with the external operational database.

4.2. Role-Based Access Control (RBAC) & Cluster Provisioning

To emulate production-grade security and role separation, cluster operations were strictly divided between service daemons and client execution. Administrative tasks were executed using the dedicated cluster service account (`hadoop`).

Listing 4: Cluster provisioning using the hadoop service account

```
# Fix ownership and permissions for daemon log directories
sudo chown -R hadoop:hadoop /home/hadoop/hadoop/logs
sudo chmod 775 /home/hadoop/hadoop/logs

# Switch to the hadoop service user and start daemons
su - hadoop
start-dfs.sh
start-yarn.sh

# Create client workspace on HDFS and assign ownership to user 'student'
hdfs dfs -mkdir -p /user/student
hdfs dfs -chown -R student:student /user/student
```

This setup ensures that system processes are isolated, and the data engineering client (the `student` user) only has access to their explicitly provisioned HDFS workspaces, preventing accidental corruption of cluster metadata. The client user then created the specific landing zones (`raw/`) for each table.

4.3. Sqoop MapReduce Ingestion Jobs

Data extraction was executed using Apache Sqoop, which translates relational data into distributed Hadoop files by launching MapReduce tasks over YARN. Four separate jobs were configured to import the `application_train` (∼307K rows), `previous_application` (∼1.67M rows), `bureau` (∼1.71M rows), and `installments_payments` (∼13.6M rows) tables.

Several critical business and technical parameters were defined in the Sqoop jobs to optimize downstream processing:

- `--as-parquetfile`: Instead of dumping raw CSVs, Sqoop was configured to immediately encode the data into Parquet format. This columnar storage format drastically reduces HDFS storage footprint and significantly accelerates read performance for the upcoming PySpark transformations.
- `-m 1`: A single mapper was used to avoid the need for primary key splitting overhead, ensuring data sequentially lands in a single file per table (`part-m-00000`).
- `--delete-target-dir`: This flag guarantees job idempotency; if the pipeline fails and needs to be rerun, Sqoop will overwrite existing directories without throwing path-collision errors.

Listing 5: Example Sqoop Import Job for application_train

```
sqoop import \
```

```
  --connect jdbc:mysql://10.29.51.13:3306/home_credit \
  --username root \
  --password rootpassword \
  --table application_train \
  --target-dir /user/student/home_credit/raw/application_train \
  --delete-target-dir \
  -m 1 \
  --as-parquetfile
```

4.4. HDFS Verification & Quality Assurance

To confirm that the MapReduce jobs completed successfully and no data was lost in transit, the HDFS landing zones were inspected using the `hdfs dfs -ls` command.

The presence of the `_SUCCESS` token alongside the data files (`part-m-00000`) acts as a systemic green light, confirming that the data was safely committed to the distributed file system and is ready for the Data Staging phase.

Figure 4: Terminal output verifying the presence of Parquet files and _SUCCESS tokens in the HDFS raw directory.

5. Phase 4: Distributed Processing & Data Selection (PySpark)

5.1. Cluster Permissions & Workspace Preparation

In a multi-tenant Hadoop ecosystem, managing permissions is critical to prevent data corruption and unauthorized access. Before launching the PySpark engine, system service accounts and client permissions were strictly configured.

Using the `hadoop` administrative account, daemon log ownership was corrected and the YARN/HDFS services were verified. Subsequently, a dedicated `staging` directory was created within HDFS and ownership was transferred to the `student` client user. This simulates a real-world enterprise environment where data engineers operate within isolated, secure namespaces.

Listing 6: Provisioning the staging workspace on HDFS

```
# Switch to hadoop service user to ensure cluster daemons are properly
    provisioned
su - hadoop
start-dfs.sh
start-yarn.sh

```

```
# Create the staging zone on HDFS and grant full access to the student
    user
hdfs dfs -mkdir -p /user/student/home_credit/staging
hdfs dfs -chown -R student:student /user/student/home_credit/staging
```

5.2. Spark Session Initialization & Data Projection

A Spark Session was initialized with Hive support enabled (`enableHiveSupport()`) to allow seamless integration with the Hadoop Metastore later in the pipeline. The raw Parquet files, previously ingested via Sqoop, were read into memory.

Instead of processing the entire raw dataset—which contains hundreds of columns that are irrelevant to our Star Schema—we applied immediate **Data Projection** (Column Selection). By explicitly selecting only the required columns and casting them to strict analytical data types (e.g., `IntegerType`, `DoubleType`), we significantly reduced the in-memory footprint and optimized the DAG (Directed Acyclic Graph) execution plan.

Listing 7: Loading raw data and enforcing strict data types

```
# 1. Application Train Projection
df_app = raw_app.select(
    F.col("SK_ID_CURR").cast("int"),
    F.col("TARGET").cast("int"),
    F.col("NAME_CONTRACT_TYPE").cast("string"),
    F.col("DAYS_BIRTH").cast("int"),
    # ... additional columns selected and casted ...
    F.col("AMT_INCOME_TOTAL").cast("double"),
    F.col("DAYS_EMPLOYED").cast("int")
)
```

Figure 5: Jupyter Notebook execution confirming the successful loading and projection of raw HDFS Parquet files.

6. Phase 5: Data Cleaning & Staging Layer

6.1. Dynamic Empty String Cleaning

When data is extracted from relational SQL databases, missing categorical values are frequently exported as empty strings (`""`) rather than true structural `NULL`s. If left unhandled, these empty strings bypass null-check functions and skew analytical results.

To dynamically resolve this, a PySpark function was engineered to iterate through the schema, detect all `StringType` fields, and convert any whitespace-trimmed empty strings

into proper `None` (NULL) values. This ensures downstream algorithms handle missingness correctly.

Listing 8: Dynamic conversion of empty strings to NULLs

```
def clean_empty_strings(df):
    string_cols = [f.name for f in df.schema.fields if isinstance(f.
        dataType, StringType)]
    for c in string_cols:
        df = df.withColumn(c, F.when(F.trim(F.col(c)) == "", None).
            otherwise(F.col(c)))
    return df

df_app = clean_empty_strings(df_app)
```

6.2. Referential Integrity: Deduplication & Null Filtering

To maintain the strict one-to-one and one-to-many relationships defined in our data model, referential integrity was enforced computationally. The PySpark `dropDuplicates()` function was applied across the primary keys (`SK_ID_CURR` and `SK_ID_PREV`) to remove any transactional anomalies. Furthermore, records missing a primary key (`isNotNull()`) were completely filtered out, as orphaned records hold no analytical value and break table joins.

6.3. Business Logic & Anomaly Correction

Raw data often contains system-generated defaults or logical impossibilities that contradict business reality. Several specific anomalies were targeted and corrected using Spark's `when/otherwise` logic:

- **Employment Outliers:** In `application_train`, a value of `365243` in `DAYS_EMPLOYED` is a known legacy system flag for "Pensioner". This was replaced with `NULL` to prevent massive statistical skewing, and their `OCCUPATION_TYPE` was synchronized to "Retired".
- **Absolute Time Metrics:** Time-based metrics (e.g., `DAYS_BIRTH`, `DAYS_EMPLOYED`, `DAYS_INSTALMENT`) were originally recorded as negative integers relative to the application date. These were converted to absolute positive values (`F.abs()`) to simplify human readability and mathematical aggregations.
- **Financial Impossibilities:** In the `bureau` dataset, credit limits (`AMT_CREDIT_SUM`) less than or equal to zero were nullified, and missing financial overdues were explicitly imputed to `0.0`.

Listing 9: Correcting known business logic anomalies

```
# Fix anomalies in Application Train
clean_app_fixed = clean_app \
    .withColumn("DAYS_EMPLOYED", F.when(F.col("DAYS_EMPLOYED") ==
       365243, None).otherwise(F.col("DAYS_EMPLOYED"))) \
    .withColumn("DAYS_BIRTH", F.abs(F.col("DAYS_BIRTH")))
```

6.4. Staging Layer Export

Once the DataFrames passed all data quality checks, they were exported to the HDFS `/staging/` zone. The output was written in **Parquet format** using `mode("overwrite")`.

Writing to a dedicated Staging Layer is a standard Data Engineering best practice; it creates an immutable, cleaned "checkpoint" in the data lake. Subsequent Feature Engineering pipelines can now directly load from this pristine staging zone without needing to re-process or re-clean the raw ingestion files.

Figure 6: Successful export of the cleaned DataFrames to the HDFS Staging Layer.

7. Phase 6: Distributed Feature Engineering & Aggregation

7.1. Architectural Objective & Grain Enforcement

In this critical phase, we engineered the predictive features mandated by our Star Schema design. Because our Fact table (`Fact_Loan`) must strictly maintain a one-to-one relationship with the customer (`SK_ID_CURR`), we could not perform direct joins with the transactional tables (`previous_application`, `installments_payments`, `bureau`).

To prevent Cartesian joins and memory overloads, we utilized PySpark's distributed `groupBy` transformations to condense millions of historical events into flattened, customer-level summaries before executing the final master join.

7.2. Primary Application Features (application_train)

We began by computing core financial and demographic ratios directly on the primary loan application record. These ratios assess the immediate affordability and collateral risk of the requested loan. To ensure pipeline stability and prevent runtime crashes, PySpark's `F.when().otherwise()` logic was utilized as a zero-division guard.

Listing 10: Computing application-level ratios with zero-division guards

```
fact_app_features = (
```

```
    clean_app_fixed
    .withColumn("DTI", F.when(F.col("AMT_INCOME_TOTAL") > 0, F.round(F.
        col("AMT_CREDIT") / F.col("AMT_INCOME_TOTAL"), 4)).otherwise(None
        ))
    .withColumn("Annuity_to_Income", F.when(F.col("AMT_INCOME_TOTAL") >
        0, F.round(F.col("AMT_ANNUITY") / F.col("AMT_INCOME_TOTAL"), 4)).
        otherwise(None))
    .withColumn("LTV", F.when(F.col("AMT_GOODS_PRICE") > 0, F.round(F.
        col("AMT_CREDIT") / F.col("AMT_GOODS_PRICE"), 4)).otherwise(None)
        )
    .withColumn("Employed_to_Age", F.when(F.col("DAYS_EMPLOYED").
        isNotNull(), F.round(F.col("DAYS_EMPLOYED") / F.col("DAYS_BIRTH")
        , 4)).otherwise(0.0))
)
```

7.3. Historical Application Aggregations

For the `previous_application` data, we performed a grouped aggregation to measure the customer's historical reliance on Home Credit and their past success rates. Metrics such as the `Approved_App_Ratio` and `Refused_App_Ratio` serve as strong behavioral indicators of past creditworthiness.

Listing 11: Aggregating historical loan applications

```
prev_features = clean_prev.groupBy("SK_ID_CURR").agg(
    F.count("SK_ID_PREV").alias("Prev_App_Count"),
    F.round(F.sum(F.when(F.col("NAME_CONTRACT_STATUS") == "Approved", 1)
        .otherwise(0)) / F.count("SK_ID_PREV"), 4).alias("
        Approved_App_Ratio"),
    F.round(F.sum(F.when(F.col("NAME_CONTRACT_STATUS") == "Refused", 1).
        otherwise(0)) / F.count("SK_ID_PREV"), 4).alias("
        Refused_App_Ratio"),
    F.round(F.avg("AMT_CREDIT"), 2).alias("Avg_Prev_Credit")
)
```

Figure 7: Output preview of the aggregated historical application features.

7.4. Two-Stage Installment Repayment Aggregation

The `installments_payments` table presented a unique business logic challenge: customers frequently make partial payments, resulting in multiple transaction records mapping to a single scheduled installment. A standard aggregation would incorrectly sum the expected installment amounts, leading to corrupted data.

To resolve this, a **Two-Stage Aggregation** was engineered:

1. **Installment Grain Consolidation:** First, we grouped by `SK_ID_CURR`, `SK_ID_PREV`, `NUM_INSTALMENT_VERSION`, and `NUM_INSTALMENT_NUMBER` to find the true scheduled amount (`max`) and the total actual amount paid (`sum`) for that specific installment.

2. **Customer Grain Aggregation:** Next, we grouped the consolidated installments by `SK_ID_CURR` to calculate overarching credit-discipline metrics, such as total days past due, the number of late payments, and the overall payment ratio.

Listing 12: Stage 2: Customer Grain Aggregation for Installments

```
inst_agg = (
    inst_level.groupBy("SK_ID_CURR").agg(
        F.round(F.sum("Days_Past_Due"), 2).alias("Total_Days_Past_Due"),
        F.sum("Is_Late").alias("Num_Late_Payments"),
        F.round(F.sum("Underpaid_Amount"), 2).alias("Total_Underpaid"),
        F.sum("AMT_PAYMENT").alias("SUM_AMT_PAYMENT"),
        F.sum("AMT_INSTALMENT").alias("SUM_AMT_INSTALMENT")
    )
)
```

7.5. External Bureau Credit Features

The `bureau` table was aggregated to quantify the applicant's leverage and delinquency behavior with external financial institutions. Features such as `Total_External_Debt` and `Max_External_Overdue` were calculated to expose severe financial distress outside of the Home Credit ecosystem.

7.6. Cross-Table Feature Derivation & Fact Construction

In the final step of the Feature Engineering pipeline, we executed a master `LEFT JOIN` sequence, attaching all aggregated historical summaries (`prev_features`, `inst_features`, `bureau_features`) onto the main `fact_app_features` DataFrame using `SK_ID_CURR` as the key.

Once unified, we derived complex cross-table composite features. For example, `Total_Overall_Debt` was computed by adding the current requested `AMT_CREDIT` to the `Total_External_Debt` from the bureau data, utilizing `F.coalesce()` to safely handle users with no external credit history.

Listing 13: Deriving cross-table composite features and validating grain

```
fact_loan = (
    fact_loan
    .withColumn("Total_Overall_Debt", F.col("AMT_CREDIT") + F.coalesce(F
        .col("Total_External_Debt"), F.lit(0.0)))
```

```
    .withColumn("External_DTI", F.when(F.col("AMT_INCOME_TOTAL") > 0, F.
        round(F.coalesce(F.col("Total_External_Debt"), F.lit(0.0)) / F.
        col("AMT_INCOME_TOTAL"), 4)).otherwise(0.0))
)

# Schema & Grain Integrity Validation
assert fact_loan.count() == fact_loan.select("SK_ID_CURR").distinct().
    count()
```

The pipeline concluded with a programmatic assertion to validate grain integrity, mathematically guaranteeing that the resulting `Fact_Loan` DataFrame maintained exactly 307,511 rows, matching the distinct count of `SK_ID_CURR`.

Figure 8: Terminal validation confirming the Fact table successfully maintained the one-to-one customer grain.

8. Phase 7: Data Warehouse Serving Layer (Star Schema)

8.1. Architectural Split: Dimension and Fact Separation

In this final phase of the data engineering pipeline, the unified, engineered dataset was transitioned into a Star Schema architecture optimized for both Machine Learning and Business Intelligence (BI) analytics. The master dataset was split into a dimension table (`Dim_Customer`) for categorical data and a central fact table (`Fact_Loan`) containing pure numerical features.

- **Dimension Table (`Dim_Customer`):** All descriptive and categorical applicant profiles (e.g., `CODE_GENDER`, `OCCUPATION_TYPE`, `NAME_EDUCATION_TYPE`) were extracted to maintain a clean numerical Fact table. This separation provides the necessary metadata for descriptive analytics and BI dashboards without cluttering the ML feeding layer.
- **Fact Table (`Fact_Loan`):** Categorical columns were dropped from the master joined table, retaining exclusively the `TARGET` label and all calculated numerical/continuous features. This makes the table mathematically ready for ingestion by predictive algorithms like XGBoost or LightGBM.

8.2. Surrogate Key Generation

To adhere to Data Warehousing best practices, a surrogate key named `loan_key` was dynamically added to the Fact table. This was achieved using PySpark's `monotonically_increasing_id()`

function, ensuring each record has a unique, auto-incrementing identifier independent of the business keys.

8.3. Physical Storage Optimization (Parquet, Snappy & Partitioning)

The finalized tables were persisted into the Hadoop Distributed File System (HDFS) using the Parquet format with Snappy compression.

- **Format & Compression:** Parquet provides columnar storage, which delivers optimal read performance for analytical queries, while Snappy compression ensures a minimal storage footprint without heavy CPU overhead.
- **Partitioning:** The `Fact_Loan` table was physically partitioned on disk by the `NAME_CONTRACT_TYPE` column. This strategy allows downstream querying engines to utilize predicate pushdown, skipping irrelevant directories and vastly accelerating query times for specific contract types.

Listing 14: Saving Data Warehouse tables with Partitioning and Compression

```
# Save Fact Table with Partitioning
(fact_loan_final.write
    .mode("overwrite")
    .format("parquet")
    .option("compression", "snappy")
    .partitionBy("NAME_CONTRACT_TYPE")
    .option("path", "/user/student/home_credit/warehouse/fact_loan")
    .saveAsTable("home_credit_dw.fact_loan"))
```

Figure 9: Jupyter Notebook output confirming the successful creation of the Data Warehouse tables in HDFS.

8.4. Hive Metastore Integration (DDL)

Finally, both tables were registered in the Hive Metastore under the `home_credit_dw` database for immediate Spark SQL and Hive querying. To illustrate the underlying schema explicitly, the equivalent Hive Data Definition Language (DDL) is provided below.

When creating partitioned external tables over existing HDFS Parquet files, the `MSCK REPAIR TABLE` command is strictly required. This command forces the Hive Metastore to scan the HDFS directory and synchronize the Spark-generated partition folders into the Hive catalog.

Listing 15: Hive DDL for the partitioned Fact_Loan table

```
CREATE DATABASE IF NOT EXISTS home_credit_dw;
USE home_credit_dw;

CREATE EXTERNAL TABLE IF NOT EXISTS fact_loan (
    loan_key BIGINT,
    SK_ID_CURR INT,
    TARGET INT,
    DAYS_BIRTH INT,
    AMT_INCOME_TOTAL DOUBLE,
    -- ... [All numerical engineered features] ...
    Current_vs_Prev_Credit DOUBLE,
    Annuity_vs_Historical_Payment DOUBLE
)
PARTITIONED BY (NAME_CONTRACT_TYPE STRING)
STORED AS PARQUET
LOCATION '/user/student/home_credit/warehouse/fact_loan'
TBLPROPERTIES ('parquet.compression'='SNAPPY');

-- Repair table to register partitions created by Spark
MSCK REPAIR TABLE fact_loan;
```

Figure 10: Verification of the `fact_loan` table and its partitions within the Hive interface.

9. Phase 8: Predictive Modeling & Machine Learning (AI)

9.1. Overview & Integration with the Data Warehouse

After successfully establishing the distributed Data Warehouse layer (Star Schema) in HDFS and Hive, this final phase bridges the data engineering pipeline with predictive modeling. The objective is to consume the cleaned, partitioned Parquet tables (`fact_loan` and `dim_customer`) to train an advanced Machine Learning model capable of predicting loan default risks (`TARGET`). This section details every procedural cell executed in the modeling notebook, explaining the rigorous technical decisions made to address real-world financial data challenges.

9.2. Cell 1: Environment Setup & Library Imports

The modeling workflow begins by importing foundational numerical, data manipulation, and visualization libraries (`numpy`, `pandas`, `matplotlib`, `seaborn`) alongside machine

learning components from `scikit-learn` and `xgboost`. Warnings are explicitly suppressed to maintain clean execution logs.

Listing 16: Importing core machine learning and evaluation libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, roc_auc_score,
    classification_report
from xgboost import XGBClassifier

import warnings
warnings.filterwarnings('ignore')
```

9.3. Cell 2: Partition-Aware Data Loading & Merging

To consume data from the Hive-partitioned warehouse without schema mismatches, a robust loading mechanism was implemented using `pathlib` and `pyarrow.dataset`:

- **Hive Partition Discovery:** The root directory of `fact_loan` contains structural partition folders split by contract type (`NAME_CONTRACT_TYPE=*`). The script recursively discovers and reads these Parquet partition files, concatenating them into a unified Pandas DataFrame while preserving partition labels.
- **Dimension Parsing:** The `dim_customer` table is loaded using explicit Hive-style partitioning.
- **Deduplication & Merging:** To eliminate duplicate entries from Spark overwrite actions, records are deduplicated based on `SK_ID_CURR` (retaining the last instance). The fact and dimension tables are then merged via an inner join on `SK_ID_CURR`, stripping duplicate suffix columns to yield a consolidated matrix of 307,511 unique applicants.

Listing 17: Loading partitioned Parquet tables and merging dataframes

```
from pathlib import Path
import pyarrow.dataset as ds

data_dir = Path("home_credit_tables")
if not data_dir.exists():
    data_dir = Path("Documentation/8_Ai Model/home_credit_tables")
```

```
7
8  fact_path = data_dir / "fact_loan"
9  fact_files = sorted(fact_path.glob("NAME_CONTRACT_TYPE=*/*.parquet"))
10
11 df_fact = pd.concat([pd.read_parquet(f, engine="pyarrow") for f in
       fact_files], ignore_index=True)
12
13 def read_partitioned_table(table_name):
14     dataset = ds.dataset(str(data_dir / table_name), format="parquet",
           partitioning="hive")
15     return dataset.to_table().to_pandas()
16
17 df_dim = read_partitioned_table("dim_customer")
18
19 df_fact = df_fact.drop_duplicates(subset=["SK_ID_CURR"], keep="last")
20 df_dim = df_dim.drop_duplicates(subset=["SK_ID_CURR"], keep="last")
21
22 df = pd.merge(df_fact, df_dim, on="SK_ID_CURR", how="inner", suffixes=("
       ", "_dup"))
23 df = df.loc[:, ~df.columns.str.endswith("_dup")]
```

9.4. Cell 3: Dataset Inspection & Identifier Dropping

The third cell inspects the schema data types and non-null counts across the 45 consolidated columns. Crucially, non-predictive identification columns—specifically the primary customer ID (`SK_ID_CURR`) and the synthetic surrogate key (`loan_key`)—are dropped. This prevents tree-based algorithms from memorizing record ordering or arbitrary surrogate mappings, ensuring the model generalizes purely on financial behavior.

Listing 18: Dropping non-predictive identifiers and surrogate keys

```
1  df.info()
2
3  columns_to_drop = ['SK_ID_CURR', 'loan_key']
4  df = df.drop(columns=[col for col in columns_to_drop if col in df.
       columns])
```

9.5. Cell 4: Exploratory Class Imbalance Analysis

A critical challenge in credit risk modeling is severe target skew. This cell analyzes the distribution of the `TARGET` variable, revealing that defaults (TARGET = 1) constitute only 8.07Failing to address this extreme imbalance would cause a naive model to achieve 91.9

Listing 19: Analyzing target class distribution

```
target_counts = df['TARGET'].value_counts()
target_pct = df['TARGET'].value_counts(normalize=True) * 100

print("Class counts:\n", target_counts)
print("\nClass percentage:\n", target_pct.round(2))
```

Figure 11: Bar plot illustrating the severe class imbalance between repaid loans and defaults.

9.6. Cell 5: Categorical Encoding (One-Hot Encoding)

Machine learning algorithms require numerical inputs. Categorical columns (such as `CODE_GENDER`, `NAME_CONTRACT_TYPE`, `OCCUPATION_TYPE`, etc.) are identified and transformed using One-Hot Encoding via `pd.get_dummies`. The `drop_first=True` parameter is enforced to avoid the dummy variable trap (collinearity), expanding the feature space from 45 to 78 dimensions.

Listing 20: Applying One-Hot Encoding to categorical attributes

```
object_columns = df.select_dtypes(include=['object', 'category']).
    columns
df_encoded = pd.get_dummies(df, columns=object_columns, drop_first=True)
print(f"Shape after encoding: {df_encoded.shape}")
```

9.7. Cell 6: Stratified Train-Test Splitting

To prevent data leakage and evaluate the model fairly, the feature matrix (X) is separated from the target label (y). The dataset is split into an 80Crucially, `stratify=y` is enforced during the split to guarantee that the minority default rate (8.07

Listing 21: Performing a stratified 80/20 train-test split

```
X = df_encoded.drop(columns=['TARGET'])
y = df_encoded['TARGET']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

9.8. Cell 7: Numerical Feature Standardization

High-variance numerical attributes (such as total income and credit amounts) can distort gradient descent and distance-based calculations. A `StandardScaler` is initialized and

fitted **strictly on the training partition** (`X_train`) and subsequently applied to transform both `X_train` and `X_test`. Fitting exclusively on training data prevents validation data leakage.

Listing 22: Standardizing numerical features without leakage

```
num_columns = X_train.select_dtypes(include=['int64', 'float64']).
    columns

scaler = StandardScaler()
X_train[num_columns] = scaler.fit_transform(X_train[num_columns])
X_test[num_columns] = scaler.transform(X_test[num_columns])
```

9.9. Cell 8: Cost-Sensitive Weighting & XGBoost Training

To counteract the 91

- **Scale Pos Weight Calculation:** The majority-to-minority ratio ($N_{\text{negative}}/N_{\text{positive}} \approx 11.39$) is calculated and passed to the `scale_pos_weight` parameter. This heavily penalizes false negatives during tree splitting, compelling the model to treat default instances with high operational importance.
- **XGBClassifier Configuration:** An optimized gradient boosting classifier is deployed with hyperparameters tuned for stability: `n_estimators=200` for progressive residual shrinkage, `max_depth=5` to prevent overfitting, `subsample=0.8` for row-level stochasticity, and `n_jobs=-1` for full CPU core utilization.

Listing 23: Calculating $\text{scale}_{p}os_{w}eightandtrainingtheXGBoostmodel$

```
pos_count = sum(y_train == 1)
neg_count = sum(y_train == 0)
balance_weight = neg_count / pos_count
print(f"Scale Pos Weight: {balance_weight:.2f}")

xgb_model = XGBClassifier(
    scale_pos_weight=balance_weight,
    n_estimators=200,
    max_depth=5,
    learning_rate=0.1,
    subsample=0.8,
    random_state=42,
    n_jobs=-1
)

xgb_model.fit(X_train, y_train)
```

9.10. Cell 9: Comprehensive Model Evaluation & Diagnostics

The final cell evaluates model performance on the unseen test set through discrimination metrics, confusion matrix breakdown, and classification reports:

- **ROC-AUC Score:** Because overall accuracy is misleading under severe class imbalance, evaluation centers on the Receiver Operating Characteristic Area Under the Curve (ROC-AUC), yielding a solid test score of `0.7172`.
- **Confusion Matrix Analysis:** Out of 61,503 test instances, the cost-sensitive weighting successfully captured 60
- **Classification Report Breakdown:** Class 1 (Default) achieves a recall of `0.60` (successfully identifying most defaulting applicants) at the trade-off of a precision of `0.15`, establishing an operational triage threshold where flagged applications undergo secondary human review.

Listing 24: Evaluating model performance via ROC-AUC and Confusion Matrix

```
y_pred_proba = xgb_model.predict_proba(X_test)[:, 1]
y_pred = xgb_model.predict(X_test)

roc_auc = roc_auc_score(y_test, y_pred_proba)
print(f"XGBoost ROC-AUC Score: {roc_auc:.4f}\n")

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.show()

print(classification_report(y_test, y_pred))
```

Figure 12: Confusion matrix heatmap and classification report evaluating the trained XGBoost model.